

RTOS Book

Mohan Arunachalam

Contents

1	RTOS	2
1.1	Fundamentals	2
1.2	Concepts and Recommendations	2
1.2.1	The Pitfalls of Dynamic Memory (malloc)	2
1.2.2	Task States and Timing Analysis	2
1.2.3	Queues: The Right Way to Pass Data	2
1.2.4	Software Timers vs. Hardware Timers	3
1.2.5	The Tick Hook & System Heartbeat	3
1.2.6	The Idle Task & Power Management	3
1.2.7	Event Groups (The “Wait for All” logic)	3
1.3	Problem solving	3
1.3.1	Odd / Even Print	3
1.3.2	The Producer-Consumer Pattern	4
1.3.3	Queue Overflow	5
1.3.4	Handling Shared Hardware (The Re-entrancy Problem)	6
1.3.5	The Deadlock Scenario	7
1.3.6	Priority Inversion:	7
1.3.7	Interrupt to Task Communication	8
2	Zephyr RTOS in Practice: Build Scalable Embedded Application	9
2.1	Introduction	9
2.1.1	What is Zephyr?	9
2.1.2	Where Zephyr Comes From?	9
2.1.3	Zephyr’s Key Characteristics	9
2.1.4	Why You Should Learn Zephyr	10
2.1.5	Zephyr from the Arduino Perspective	10
2.1.6	Why Arduino Users Should Learn Zephyr	10
2.1.7	Zephyr Architecture Overview	10
2.1.8	How Zephyr Projects Are Structured	11
2.1.9	An Introduction to Kconfig / prj.conf	12
2.1.10	An Introduction to Devicetree	12
2.1.11	West Fundamentals	14
2.1.12	Zephyr logging	14
2.2	Examples	15
2.2.1	Blinky	15

Chapter 1

RTOS

1.1 Fundamentals

1.2 Concepts and Recommendations

1.2.1 The Pitfalls of Dynamic Memory (malloc)

In a standard PC application, you use malloc and free without much thought. In an RTOS, this is often forbidden or highly discouraged.

- **Non-Determinism:** malloc takes a variable amount of time to find a free block. In a real-time system, you need to know exactly how long an operation takes.
- **Fragmentation:** Over time, allocating and freeing small chunks of memory creates “holes.” Eventually, you might have 10KB of free memory total, but not a single continuous 2KB block, causing the system to crash.
- **Solution:** Most RTOS developers use Static Allocation or Memory Pools (fixed-size blocks) to ensure memory is always available and allocation time is constant.

1.2.2 Task States and Timing Analysis

Understanding the State Transition Diagram is crucial. A task isn’t just “running” or “not running.”

- **Ready:** The task is prepared to run but a higher-priority task is currently using the CPU.
- **Running:** The task is currently executing instructions.
- **Blocked/Waiting:** The task is waiting for an event (a timer, a semaphore, or a queue).
- **Suspended:** The task is manually paused and won’t run until another task “resumes” it.

The Interview Question: “What happens if a high-priority task never enters the Blocked state?”

The Answer: It “starves” the system. Lower-priority tasks (including the Idle task which often handles power saving) will never get a single CPU cycle.

1.2.3 Queues: The Right Way to Pass Data

While Semaphores are great for signals, Queues are the gold standard for data.

- **Copy by Value vs. Copy by Reference:** Small data (like a sensor integer) is copied directly into the queue. For large data (like a 1MB image buffer), you put the pointer to the data in the queue.
- **Thread Safety:** Queues are internally protected by the RTOS. You don’t need to wrap a queue in a Mutex; the RTOS handles the locking for you.

1.2.4 Software Timers vs. Hardware Timers

- **Hardware Timers:** Use high-precision peripherals to trigger ISRs. Best for microsecond-level accuracy (e.g., PWM or high-speed sampling).
- **Software Timers:** Managed by the RTOS “Timer Task.” They are easier to use and don’t consume hardware resources, but their accuracy is limited by the RTOS Tick Rate (usually 1ms). If your tick is 1ms, you cannot have a software timer trigger every 0.5ms.

1.2.5 The Tick Hook & System Heartbeat

Every RTOS relies on a hardware timer (usually called the SysTick in ARM Cortex-M) to create a periodic interrupt.

- **The Problem:** You need to perform a small action (like blinking a status LED or updating a watchdog timer) every 1ms, but you don’t want to create a whole new task for it.
- **The Concept:** The Tick Hook is a callback function inside the RTOS kernel that executes every single time the system tick interrupt occurs.
- **The Challenge:** Since this runs inside an ISR context, you must be extremely careful. If the Tick Hook takes too long, the whole system drifts in time.

1.2.6 The Idle Task & Power Management

When no tasks are in the “Ready” state (e.g., everyone is blocked waiting for a sensor or a timer), the RTOS doesn’t just stop. It runs the Idle Task.

- **The Problem:** A battery-powered device needs to last for months. If the CPU is always “Running” (even if just in a loop), the battery will die in days.
- **The Concept:** Tickless Idle. Instead of waking up every 1ms to check the scheduler, the RTOS calculates how long it can safely “sleep” until the next scheduled event and puts the CPU into a Deep Sleep mode.
- **The Interview Question:** “What is the priority of the Idle Task, and why?”
- **The Answer:** It is always Priority 0 (the lowest). If it were higher, it would block every other task from running.

1.2.7 Event Groups (The “Wait for All” logic)

Sometimes, a task needs to wait for multiple things to happen before it can proceed. For example: “Start the Motor only if (Button is Pressed) AND (Temperature is OK) AND (Safety Cover is Closed).”

- **The Problem:** Using three separate semaphores is inefficient and can lead to complex code.
- **The Concept:** Event Groups / Event Flags.

1.3 Problem solving

1.3.1 Odd / Even Print

- **Problem:** Create a two tasks where one task print odd number and another prints even number but while printing the numbers should be in 1, 2, 3, 4, ... and So on.

Code

```
1 SemaphoreHandle_t semOdd = 1; // Start with 1 so Odd Task runs first
2 SemaphoreHandle_t semEven = 0; // Start with 0 so Even Task blocks
3
4 void Task_Odd(void) {
5     int count = 1;
6     while(1) {
7         take(semOdd);          // Wait for turn
8         print(count);
```

```

9         count += 2;
10        give(semEven);           // Signal Even task
11    }
12}
13
14void Task_Even(void) {
15    int count = 2;
16    while(1) {
17        take(semEven);           // Wait for turn
18        print(count);
19        count += 2;
20        give(semOdd);           // Signal Odd task
21    }
22}

```

1.3.2 The Producer-Consumer Pattern

- **The Problem:** Task A reads data from a sensor at a high frequency and places it in a shared circular buffer. Task B processes that data and sends it over UART.
- **The Challenge:** How do you ensure Task B doesn't read an empty buffer and Task A doesn't overwrite data that Task B hasn't processed yet? Which RTOS primitive (Binary Semaphore, Counting Semaphore, or Mutex) is best for signaling Task B that data is ready?
- **Concept:** Use counting semaphores to track empty/full slots and a mutex for data integrity.

Code:

```

1 Semaphore_t semEmpty = BUFFER_SIZE; // Tracks available space
2 Semaphore_t semFull = 0;             // Tracks available data
3 Mutex_t      bufferMutex;           // Protects the buffer indices
4
5 void Producer(void) {
6     while(1) {
7         data = read_sensor();
8
9         take(semEmpty);           // Wait if buffer is full
10        lock(bufferMutex);
11
12        buffer[in] = data;
13        in = (in + 1) % SIZE;
14
15        unlock(bufferMutex);
16        give(semFull);           // Signal Consumer that data is ready
17    }
18}
19
20 void Consumer(void) {
21     while(1) {
22         take(semFull);           // Wait if buffer is empty
23         lock(bufferMutex);
24
25         data = buffer[out];
26         out = (out + 1) % SIZE;
27
28         unlock(bufferMutex);
29         give(semEmpty);         // Signal Producer that space is free
30
31         process(data);

```

```
32     }
33 }
```

1.3.3 Queue Overflow

- **The Problem:** A High-Priority Sensor Task is producing data faster than a Low-Priority Display Task can process and render it. Because the queue has a fixed size, it eventually hits its limit (becomes full).
- **The Challenge:** If the High-Priority task tries to send data to a full queue, it faces a dilemma:
 - **Block:** If it waits for space, the high-priority task stops running, potentially missing critical sensor samples (Timing Jitter).
 - **Discard:** If it skips the data, you lose information.
 - **Overwrite:** If it forces the data in, it might corrupt the message sequence.

The challenge is maintaining system responsiveness without losing the “freshest” data.

- **Concept:** Ping Pong Buffer or Double buffering

Code

```
1  #define BUF_SIZE 100
2
3  // Two physical memory areas
4  float Buffer0[BUF_SIZE];
5  float Buffer1[BUF_SIZE];
6
7  // Signals to tell the Display task which buffer is ready
8  SemaphoreHandle_t semBuffer0Ready;
9  SemaphoreHandle_t semBuffer1Ready;
10 // Signal to tell Sensor task a buffer is empty and safe to use
11 SemaphoreHandle_t semBufferProcessed;
12
13 void Sensor_Task(void) {
14     float *currentBuffer = Buffer0;
15     int index = 0;
16
17     while(1) {
18         currentBuffer[index++] = Read_Sensor();
19
20         if (index >= BUF_SIZE) {
21             // Buffer is full!
22             if (currentBuffer == Buffer0) {
23                 give(semBuffer0Ready); // Tell Display to process Buffer 0
24                 currentBuffer = Buffer1; // Immediately switch to Buffer 1
25             } else {
26                 give(semBuffer1Ready); // Tell Display to process Buffer 1
27                 currentBuffer = Buffer0; // Immediately switch to Buffer 0
28             }
29             index = 0;
30
31             // Wait until the Display task has finished at least ONE of the buffers
32             // This provides the 'Zero Data Loss' protection
33             take(semBufferProcessed);
34         }
35         TaskDelayMs(1);
36     }
37 }
38
39 void Display_Task(void) {
```

```

40 while(1) {
41     // Check if Buffer 0 is ready
42     if (take(semBuffer0Ready, 0) == SUCCESS) {
43         Render_Graph(Buffer0, BUF_SIZE);
44         give(semBufferProcessed); // Signal that Buffer 0 is now free
45     }
46     // Check if Buffer 1 is ready
47     else if (take(semBuffer1Ready, 0) == SUCCESS) {
48         Render_Graph(Buffer1, BUF_SIZE);
49         give(semBufferProcessed); // Signal that Buffer 1 is now free
50     }
51 }
52 }

```

1.3.4 Handling Shared Hardware (The Re-entrancy Problem)

- **The Problem:** Two different tasks (High Priority and Low Priority) both need to write log messages to the same SPI Flash memory.
- **The Challenge:**
 - If the Low Priority task is mid-write and the High Priority task preempts it and tries to write to SPI, the hardware state will crash. How do you protect this shared resource?
 - Why would a Mutex be preferred over a simple Global Boolean flag?
 - * **Atomicity:** Checking a flag and then setting it is a “Read-Modify-Write” operation. In assembly, this is multiple instructions. A task could check the flag, see it’s false, and get preempted before it sets it to true. Another task could then run, see the same false flag, and you now have two tasks accessing the same hardware simultaneously. A Mutex uses hardware-level atomic instructions (like Test-and-Set) to ensure this cannot happen.
 - * **Blocking vs. Polling:** With a boolean flag, a task has to “spin” (loop constantly) to check the flag, wasting CPU cycles. A Mutex puts the task into a Blocked state, allowing other tasks to run. The RTOS scheduler only wakes the task up when the Mutex becomes available.
- **Concept:** Protecting a non-reentrant peripheral using a Mutex. Mutex is preferred over a simple global boolean because priority inversion is handled by mutex using priority inheritance.

Code:

```

1  Mutex_t spiMutex;
2
3  void Log_Data(char* msg) {
4      lock(spiMutex);      // Only one task can own the SPI bus
5
6      SPI_Select_Chip();
7      SPI_Write(msg);
8      SPI_Deselect_Chip();
9
10     unlock(spiMutex);    // Release for other tasks
11 }
12
13 void Task_HighPriority(void) {
14     Log_Data("High Priority Alert!");
15 }
16
17 void Task_LowPriority(void) {
18     Log_Data("Routine system log.");
19 }

```

1.3.5 The Deadlock Scenario

- **The Problem:** Task 1 needs Resource A then Resource B. Task 2 needs Resource B then Resource A.
- **The Challenge:** Describe the “Deadly Embrace” where both tasks end up waiting forever. What are two architectural ways to prevent this in a real-time system?
 - **Resource Hierarchy (Lock Ordering):** Assign a numerical “rank” to every resource (e.g., SPI=1, I2C=2, UART=3). Rule: A task can only acquire a resource with a higher rank than the ones it already holds. This makes it mathematically impossible to form a circular wait.
 - **Timeout-Based Acquisition:** Instead of waiting forever (wait_forever), use a timeout (e.g., take_mutex(lock, 100ms)). If the task can’t get all the resources it needs within the time limit, it must release all its held locks and try again later. This breaks the “Hold and Wait” condition of deadlocks.
- **Concept:** To prevent circular wait, tasks must acquire resources in the exact same numerical order.

Code:

```
1  Mutex_t Mutex_A; // Resource 1
2  Mutex_t Mutex_B; // Resource 2
3
4  void Task_Safe(void) {
5      while(1) {
6          // ALWAYS take A before B
7          lock(Mutex_A);
8          lock(Mutex_B);
9
10         // Access shared resources
11
12         unlock(Mutex_B);
13         unlock(Mutex_A);
14     }
15 }
```

1.3.6 Priority Inversion:

- **The Problem:** A High-priority task is waiting for a Mutex held by a Low-priority task. A Medium-priority task starts running, preventing the Low-priority task from finishing and releasing the Mutex.
- **The Challenge:** How does the “Priority Inheritance” protocol solve this? Can you explain the difference between Priority Inheritance and Priority Ceiling?
 - **Priority Inheritance:** The priority of the lower-priority task holding the lock is boosted only when a higher-priority task tries to acquire that same lock. It is “reactive.”
 - **Priority Ceiling:** Every resource (Mutex) is assigned a “Ceiling” priority (the priority of the highest task that could ever use it). As soon as any task grabs that Mutex, its priority is immediately boosted to that Ceiling, regardless of whether a higher-priority task is actually waiting. This is “proactive” and effectively prevents deadlocks as well.
- **Concept:** Illustrating how a Mutex with inheritance automatically fixes the “Unbounded” inversion.

Code

```
1  /*
2   RTOS Settings: Mutex must be created with Priority Inheritance ENABLED.
3   Low Priority (LP) holds Mutex.
4   High Priority (HP) wants Mutex.
5   LP is boosted to HP's priority until it unlocks.
6  */
7
8  void Task_Low(void) {
9      lock(mutexInherit);
```

```

10 // Even if Task_Medium tries to preempt here,
11 // it fails because Task_Low is now "High Priority"
12 unlock(mutexInherit); // Priority returns to Low
13 }

```

1.3.7 Interrupt to Task Communication

- **The Problem:** An external GPIO interrupt (ISR) triggers every time a button is pressed. You want to toggle an LED in a Task.
- **The Challenge:** Why shouldn't you toggle the LED or perform heavy logic directly inside the ISR? Explain the concept of "Deferred Interrupt Processing" using a semaphore or task notification.
 - Why avoid heavy logic/LED toggling in ISRs?
 1. **Jitter:** While an ISR is running, other interrupts of the same or lower priority are blocked. This creates "latency" for other parts of the system.
 2. **Stack Overflow:** ISRs often use a limited system stack. Complex logic or printf calls can easily crash the system.
 3. **Non-deterministic behavior:** Many RTOS functions (like delay or lock_mutex) can block. You cannot block inside an ISR, as there is no "task context" to switch away from.
 - **Deferred Interrupt Processing:** Instead of doing the work in the ISR, the ISR simply "gives" a semaphore or a task notification and exits. The RTOS then sees a high-priority "Worker Task" is now ready to run. The worker task wakes up, does the heavy lifting (like writing to an SD card or toggling an LED), and then goes back to sleep. This ensures the hardware interrupt is cleared in microseconds, keeping the system responsive.

Concept: Keep the ISR short by offloading logic to a task using a binary semaphore.

Code

```

1 Semaphore_t semButton;
2
3 // ISR (Interrupt Service Routine)
4 void GPIO_ISR(void) {
5     // Keep this extremely fast!
6     give_from_ISR(semButton); // Wake up the task
7 }
8
9 // Processing Task
10 void Task_ButtonHandler(void) {
11     while(1) {
12         take(semButton); // Blocks here until button is pressed
13         // Perform heavy work (e.g., Debouncing, Network requests)
14         Toggle_LED();
15         printf("Button Pressed!\n");
16     }
17 }

```

Chapter 2

Zephyr RTOS in Practice: Build Scalable Embedded Application

2.1 Introduction

2.1.1 What is Zephyr?

- A scalable, open-source RTOS for embedded systems.
- Supports microcontrollers, IoT devices, wearables, and edge systems.
- Backed by a large industry consortium through the Linux Foundation.
- Actively used in real-world commercial products.



A modern, industry-backed RTOS for real-world IoT products.

2.1.2 Where Zephyr Comes From?

- Born from the merger of two RTOS projects: Rocket RTOS and Wind River's Microkernel.
- Adopted by the Linux Foundation in 2016.
- Designed as a modern alternative to legacy RTOS systems with stronger security, modularity, and tooling.

Year	Event
2001	Wind River microkernel introduced
2015	Rocket RTOS open-sourced
2016	Zephyr adopted by Linux Foundation
2019	Major modularity improvements
2021+	Rapid ecosystem growth



A modern, unified RTOS built to replace fragmented legacy systems.

2.1.3 Zephyr's Key Characteristics

- Highly modular kernel (choose only what your application needs).
- Memory-safe design with strong security primitives.
- Unified device driver model with devicetree support.
- Multi-architecture: ARM, RISC-V, Xtensa, x86, ARC, MIPS.

- First-class networking: BLE, Wi-Fi, Ethernet, Thread, Matter.



A scalable, secure RTOS platform with unified APIs and rich networking.

2.1.4 Why You Should Learn Zephyr

- Industry demand for engineers who understand modern RTOS design.
- Works across many chips and vendors, so your skills transfer easily.
- Excellent documentation, CI-tested code, and long-term stability.
- Powerful abstraction layers reduce vendor lock-in.
- Ideal for commercial IoT products that require security + maintainability.



Zephyr gives you portable, in-demand skills for modern IoT products.

2.1.5 Zephyr from the Arduino Perspective

- Arduino is easy to use and ideal for beginners, but not designed for complex RTOS tasks.
- Arduino “sketches” run a single, long loop with limited concurrency.
- Peripheral handling is simplified, but lacks structured device drivers found in Zephyr.
- Memory management is minimal, making larger systems harder to scale safely.
- No built-in RTOS features like threads, scheduling, IPC, or power-aware execution.
- Zephyr offers a professional-grade environment while still supporting many Arduino-class MCUs.



Arduino is great for quick prototyping; Zephyr offers structure.

2.1.6 Why Arduino Users Should Learn Zephyr

- Move from single-loop programs to real multitasking.
- Learn industry-standard device driver models.
- Gain experience with networking stacks like Wi-Fi, BLE, Thread, and Matter.
- Understand how commercial IoT firmware is structured.



Zephyr upgrades your skills to industry-level embedded engineering.

2.1.7 Zephyr Architecture Overview

- Modular RTOS built around a small, efficient microkernel.
- Hardware abstraction through drivers and devicetree.
- Subsystems for networking, sensors, storage, Bluetooth, power management, and more.
- Clear separation between kernel, drivers, and user application code.

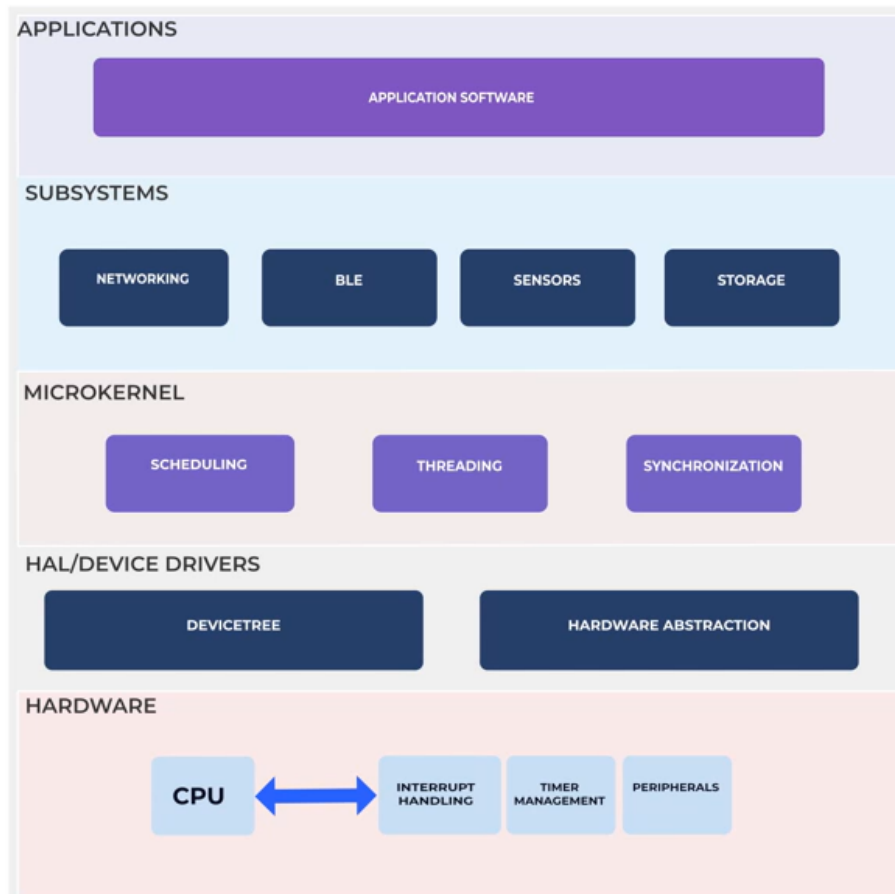


Figure 2.1: Zephyr Architecture Overview



Zephyr layers a small kernel with selectable modular services.

2.1.8 How Zephyr Projects Are Structured

- Standard layout: `src/` , `prj.conf` , `CMakeLists.txt` , and optional Devicetree overlays.
- `prj.conf` controls compile-time configuration through Kconfig.
- Devicetree overlays describe hardware, buses, and device connections.
- Build system uses CMake and `west` to generate the final executable.

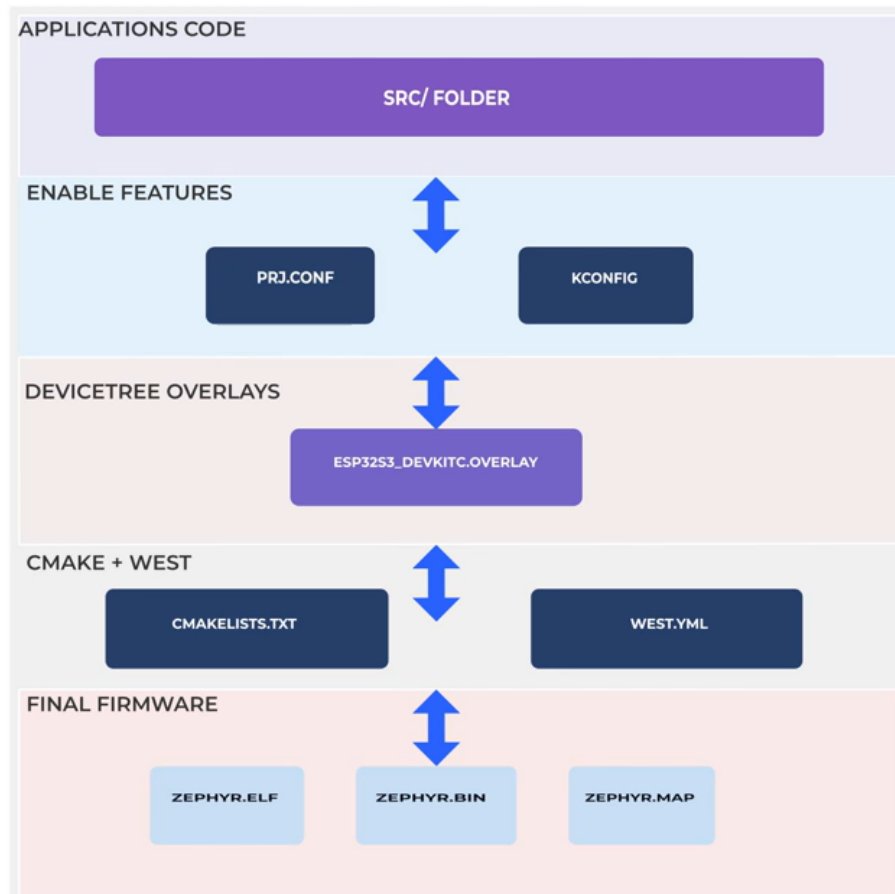


Figure 2.2: Zephyr Project Structure



Zephyr projects have clear app directories plus shared modules.

2.1.9 An Introduction to Kconfig / `prj.conf`

- `prj.conf` sets Kconfig symbols that shape features, drivers, and build options.
- Kconfig enforces dependencies and provides defaults for each board and module.
- Overlay configs (e.g., `boards/*.conf`) fine-tune per-board tweaks.



Kconfig is Zephyr's feature control system for your app.

2.1.10 An Introduction to Devicetree

- Devicetree describes what hardware exists and how it is connected.
- Board and SoC definitions live in `.dts` and `.dtsi` files.
- Applications customize hardware using Devicetree overlays.
- Zephyr generates headers and macros from Devicetree at build time

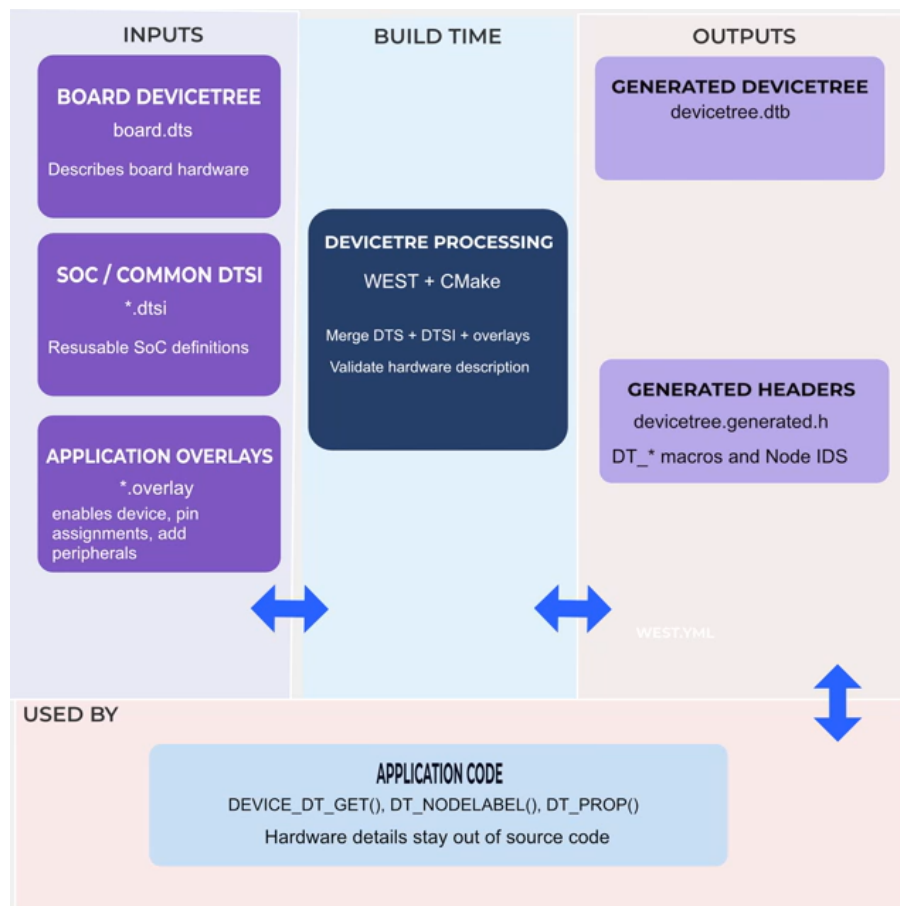


Figure 2.3: DeviceTree



In Zephyr the devicetree is the single source of truth for your hardware

2.1.11 West Fundamentals

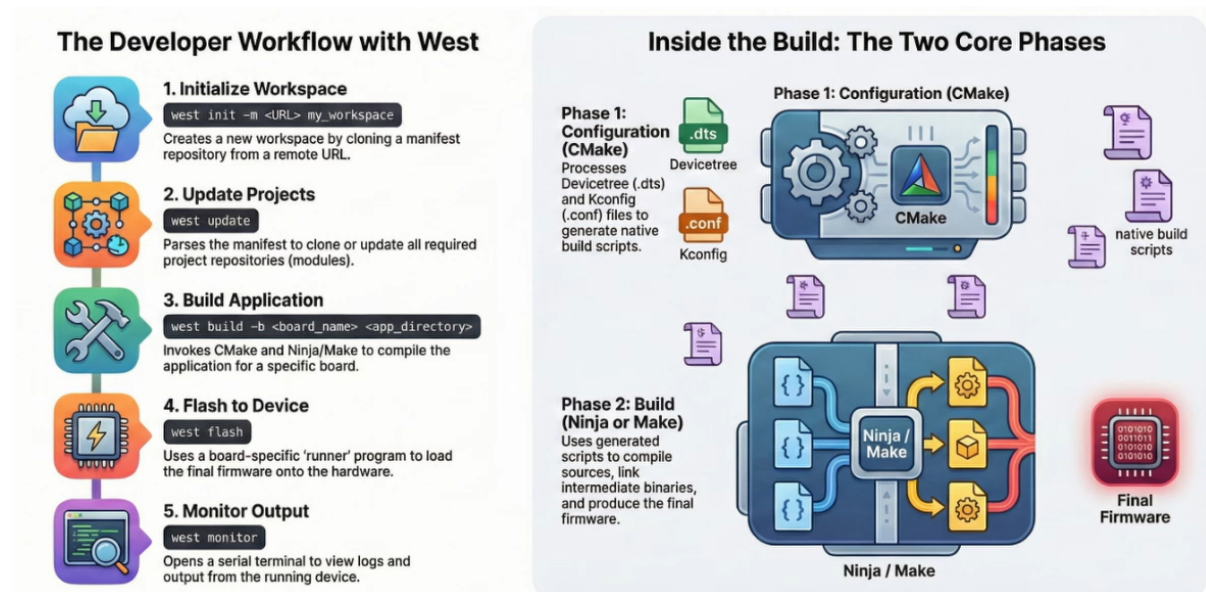


Figure 2.4: Zephyr build process: Command to Firmware

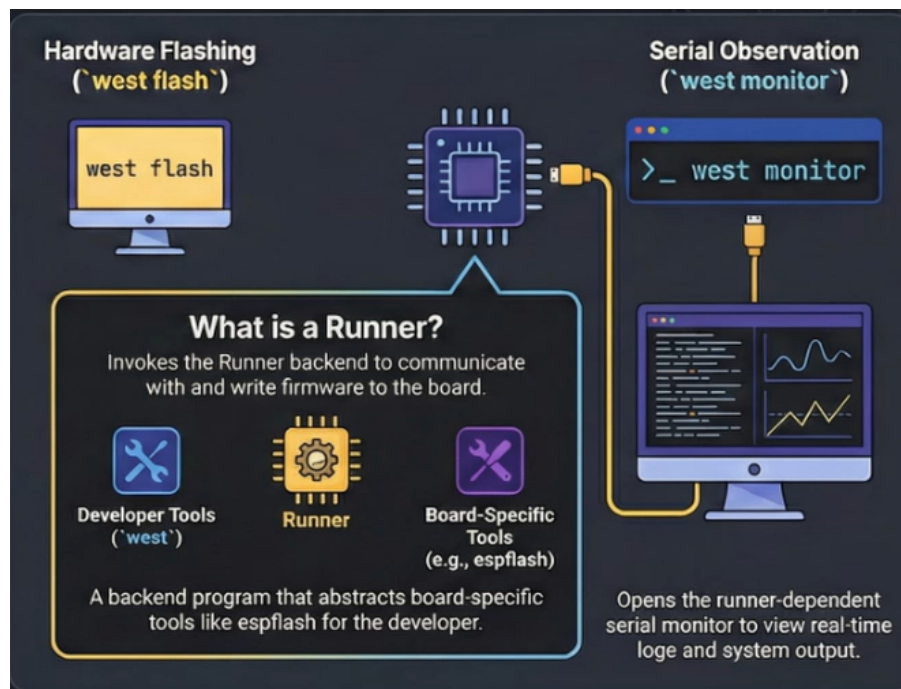


Figure 2.5: West Runners

2.1.12 Zephyr logging

- Zephyr logging provides leveled logs (error to debug) with deferred processing.
- Backends include RTT(Real-Time Transfer), UART and others. We can choose per board and bandwidth needs.
- Configure levels and formatting via `CONFIG_LOG_*` options in `prj.conf`.

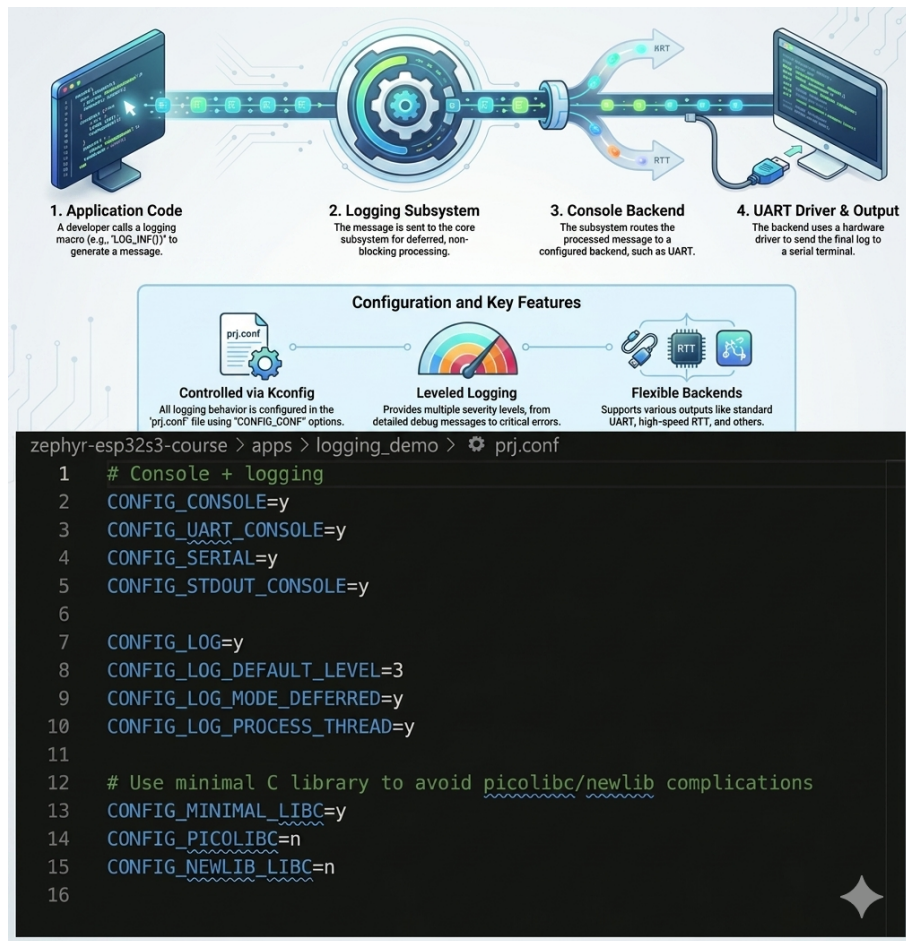


Figure 2.6: Zephyr logging

2.2 Examples

2.2.1 Blinky